```
nonce          = payload[0:CRYPTO_AEAD_NONCE_LENGTH_BYTES]
ciphertext/MIC = payload[CRYPTO_AEAD_NONCE_LENGTH_BYTES:]
```

2. The client SHALL try to decrypt the message with the key associated to the server being tested. When a client has multiple associated servers, the client iterates through this process for the keys associated to each potential server until the keys associated to all possible servers fail or a successful key is found.

```
Success, Plaintext = Crypto_AEAD_DecryptVerify(
     K = Key,
     C = ciphertext/MIC,
     A = "",
     N = nonce)
```

   a. If the decryption succeeds, the device associated with the key is identified as the server checking in. The client SHALL continue from step 3.

   b. If the decryption fails, the client SHALL move on to the next candidate server key per step 2, or discard the message if there are no remaining registered server keys to try.

3. From the plaintext, the client SHALL retrieve the Check-In Counter and the application data if present.

4. The client SHALL verify that the received nonce is valid by calculating it as described in Section 4.22.3.1, “Nonce Generation” from the received Check-In Counter and the key. The counter value retrieved in the previous step is already in the correct encoding.

   a. If the calculated nonce matches the received nonce, the client SHALL continue from step 5.

   b. If the calculated nonce doesn’t match the received nonce, the client SHALL discard the message.

5. The client SHALL verify that the received Check-In Counter is valid as described in Section 4.22.3.3, “Client Counter Validation”. The Check-In Counter is converted into a unsigned 32-bit integer.

   a. If the received counter value is valid, the client SHALL continue from step 6.

   b. If the received counter value is invalid, the client SHALL discard the message.

6. The client SHALL store the offset between the received Check-In Counter value and the stored starting value of the Check-In Counter. The subtraction SHALL be done as unsigned integers mod 2^{32}. The client SHALL continue from step 7.

7. The client SHALL verify if a key refresh is necessary as described in Section 4.22.3.4.1, “Key Refresh Validation”.

   a. If a key refresh is necessary, the client SHALL trigger an attempt to refresh the key, unless the client is already in the process of refreshing this specific key for this specific server. The client SHALL continue from step 8 whether the key refresh succeeds or not.

   b. If a key refresh is not necessary, the client SHALL continue from step 8.